

PROJECT HANDBOOK / LOCAL MVP

Codebase Doctor Complete guide

Repository understanding. Reviewed repairs. Evidence you can inspect.

Delivery and acceptance

The local implementation, portfolio-inspired UI and documentation are delivered. Static scanning and local checks passed. Real Docker execution, live Groq repairs and GitHub publication remain unverified until the required environment and credentials are available.

Inside this guide

- Getting started and operation
- Security and data boundaries
- Architecture and API
- Milestones and validation
- Report contract
- Complete folder tree and per-file responsibilities

Codebase Doctor

Diagnose. Repair. Verify. A local Java/Spring workbench with repository analysis, a Groq tool loop, reviewable repairs, and a report explaining what changed.

The UI follows the owner's portfolio direction: restrained dark/light colors, large typography, numbered sections, rounded controls, and subtle motion. It shows actual job events, source evidence, output and diffs.

What works

- Public GitHub repository and issue intake, pinned to a source commit, with bounded archive validation.
- JavaParser-based Java/Spring structure, Maven/Gradle metadata, layers, dependencies and evidence-backed findings.
- Saved jobs, progress over SSE, cancellation, overview, architecture, findings, plan, terminal, diff and documentation panels.
- Groq planning and bounded repair tools with approval bound to the exact plan and allowed file paths.
- A Docker runner that refuses execution unless rootless Docker, enforced resource controls and the trusted helper image are available.
- Markdown and printable HTML reports: repository overview, principal change, per-file explanation, baseline/final checks and remaining concerns.
- Separately approved GitHub Git Data API publication to a new doctor branch, with a draft PR selected by default.

Validation boundary: live public-repository scanning and report downloads are verified. Automated tests exercise repair and publishing with test doubles. Actual Docker execution, a live Groq repair and GitHub publication are not verified on this PC: Docker access and provider credentials were unavailable. See validation (docs/VALIDATION.md). This is a single-user local MVP, not a verified public multi-user service.

Run locally

Requirements: Java 21, Maven 3.9+, Node 20.9+ (Node 22 recommended), npm, Python 3. Docker and model keys are optional for static analysis.

```
./scripts/setup-local.sh  
cd frontend  
npm ci --ignore-scripts  
cd ..  
./scripts/dev.sh
```

For a compiled UI preview, build the frontend first, then run `./scripts/dev.sh --production` from the project root.

Open **<http://127.0.0.1:3000>**. The exact origin is checked; the API listens on 127.0.0.1:8080. Ctrl+C stops both services. Maven dependencies are cached in `.runtime/m2`.

Setup creates ignored `.env` and `frontend/.env.local` with a random server token and mode 0600. Edit `.env` as data; it is never executed as shell code. To enable planning, set `GROQ_API_KEY` and `GROQ_MODEL` (default `openai/gpt-oss-20b`). Restart the supervisor after changing settings. Repository context, issue

text and tool observations are sent to Groq when configured; inspect your public repository for embedded secrets first. No OpenAI, Claude or Codex API key is required.

For repairs, follow sandbox setup (docs/USER_GUIDE.md#enable-repository-execution). For optional publishing, set a fine-grained GITHUB_TOKEN restricted to the intended repository with Contents read/write and Pull requests read/write. Do not paste keys into the UI. A separate publication review shows the destination, branch, diff and verification before creating GitHub objects.

Demo workflow

1. Paste a public Java repository URL. Choose Doctor Scan or enter an issue/objective.
2. Review stack, architecture and findings. Without Groq, the run ends with a read-only report.
3. With Groq, review the proposed plan and its exact file scope. Approve only after sandbox readiness is confirmed.
4. Follow real edit/build/test events, then inspect the diff and baseline/final results.
5. Download Markdown or HTML documentation. HTML can be opened locally and printed to PDF.
6. Optionally review and approve publication to the new doctor branch and draft PR.

The intentionally broken token-expiry example is in examples/broken-spring-app (examples/broken-spring-app/README.md). Do not run that fixture on the host as part of agent verification.

Architecture

```
Browser -> guarded Next.js proxy -> Spring Boot API
|-> pinned GitHub source snapshot
|-> Java/Spring analyzer
|-> Groq + allowlisted tools
|-> isolated offline Docker verification
|-> saved evidence + Markdown/HTML report
|-> approved GitHub branch/draft PR
```

Source is stored as bounded data snapshots rather than a host Git checkout. Edits are exact patches to the snapshot; a branch name is reserved at approval and the real remote branch is created only during separately approved publication. Exported unified diffs use full-file replacement hunks. This design avoids host Git hooks and repository command execution. History, submodules and Git LFS hydration are outside this MVP.

Checks and documentation

```
./scripts/test.sh

# While the app is running:
python3 scripts/smoke.py

# Read-only sandbox diagnostics:
./scripts/preflight.sh
```

Read user guide (docs/USER_GUIDE.md), security (docs/SECURITY.md), architecture (docs/ARCHITECTURE.md), API (docs/API.md), file guide (docs/FILE_GUIDE.md), report format (docs/report-format.md) and milestone status (docs/MILESTONES.md). PDFs are saved in milestone-reports/; PROJECT_CONTEXT.md (PROJECT_CONTEXT.md) is the handoff for future work.

Docker Compose

`docker compose up --build` runs the UI/API for static analysis with a named data volume and no host mounts or Docker socket. Compose does **not** enable repository execution. Native development with a reviewed rootless Docker context is the execution path. Compose configuration validates; image builds/startup were not run in this session.

Current limits and next work

Public, small repositories only; 3,000 files, 24 MiB expanded, 1 MiB per file and 20 local jobs. Parsing inspects a bounded Java/config subset. Root build files are required for execution; nested standalone projects need a future module selector. Java 21 is the sandbox runtime. Dependencies must already exist in its reviewed offline cache; arbitrary Maven projects and most Gradle projects may fail offline. These are reported failures, never silent host/network fallbacks.

Next priorities are runtime isolation tests on a dedicated rootless VM, a live Groq fixture repair, broader reviewed dependency caches, and dependency-pinned release images. No accounts, teams, billing, non-Java repair, public deployment or automatic merging are included. Read security and context before contributing. No open-source license has been selected; choose one before public distribution.

User guide

Start and scan

Run the setup and development commands in the root README. Open <http://127.0.0.1:3000>. Environment shows actual readiness. Enter a repository root URL such as <https://github.com/spring-guides/gs-rest-service>. Doctor Scan permits an empty objective. Solve an issue requires issue text, an objective, or a same-repository GitHub issue URL.

The repository must be public. Intake pins the default branch commit and retrieves its archive. Invalid hosts, credential-bearing URLs, traversal and symlinks are rejected. No scripts run during ingestion or static analysis. GitHub rate limits or unavailable archives produce a failed run with a report; retry by starting a new run.

Overview lists detected stack, important files and modules. Architecture lists parsed symbols and source-level relationships. Findings are deterministic observations or suspected concerns with evidence; neither a clean scan nor a completed job guarantees defect-free code. Plan contains the model's proposed changes. Activity and Terminal show actual recorded operations/output. Diff shows full-file unified hunks and per-file selection. Documentation explains the repository and changes and provides downloads.

Enable Groq

Add `GROQ_API_KEY` and `GROQ_MODEL` to the ignored root `.env`, then restart. Default model: `openai/gpt-oss-20b`; select an available model supporting local tool calls if account access changes. The provider uses <https://api.groq.com/openai/v1/chat/completions>. See Groq tool documentation (<https://console.groq.com/docs/tool-use/overview>).

The application sends bounded repository metadata, selected readable source, issue text, findings and tool observations to Groq. Filename rules and known-secret redaction reduce accidental leakage but are not a comprehensive secret scanner. Planning can consume provider quota. Missing keys leave static analysis available; invalid keys/rate limits stop reasoning with a visible error. There is no automatic provider switch or billing operation.

Enable repository execution

Use a dedicated VM for unfamiliar repository code. Configure a rootless Docker engine yourself using Docker's official guide (<https://docs.docker.com/engine/security/rootless/>). This app never uses sudo, modifies socket permissions or installs the engine. Rootless, seccomp, cgroup v2 and systemd cgroup enforcement are required; an unrestricted rootful daemon is rejected.

Review `docker/sandbox.Dockerfile` and `docker/guard.py`, then build the trusted image from the project root:

```
docker build -f docker/sandbox.Dockerfile -t codebase-doctor-sandbox:local .  
./scripts/preflight.sh
```

Image construction needs network for official tool distributions and reviewed Maven cache dependencies. It does not execute the broken example's source: cache preparation uses a trusted generated passing test. Repository execution later has **no network**. The image contains Java 21, Maven, Gradle and the immutable helper. Its seed cache targets the bundled Spring Maven example; arbitrary dependencies are not preloaded. Gradle projects usually need a reviewed offline cache extension. Change trusted cache inputs and rebuild; never enable networking for untrusted builds as a shortcut.

A plan is needed before source edits. Approval is bound to the current source revision, objective and plan. Only listed files may change. Changing scope requires a new run. Builds/tests use fixed offline commands in new transient containers. Build scripts remain executable code inside the sandbox; controls reduce exposure but cannot guarantee protection against all kernel/runtime defects. See security documentation.

Review, cancel and retry

Use Cancel for queued, running or awaiting-approval jobs. The app interrupts work and removes job containers; uncertain cleanup is retried. Review any recorded partial edits in the final report. Interrupted jobs do not resume automatically after a server restart, and old approvals are cleared. Start a fresh analysis to retry.

A COMPLETED status means the workflow attempt ended. Read separate build/test statuses; failed, missing, skipped, unsupported and unrun checks remain explicit. A zero test exit code without usable reports is not treated as passing tests. Model summaries are explanations, not evidence of correctness.

Publish after review

Set a fine-grained GitHub token restricted to your intended repository, with Contents read/write and Pull requests read/write. Review Diff, Terminal and Documentation, then use Review publication. Confirm the exact repository, doctor branch, changed files and verification. A separate checkbox authorizes GitHub writes; draft PR is on by default. Publication may still be possible with failed verification, so the UI calls attention to those results.

The server checks the reviewed digest, persisted workspace and current default-branch head. It creates blobs, a tree, a commit and a new `codebase-doctor/<job-id>` reference. It never updates `main/master/develop`, force-pushes or merges. If the default branch moved, start a new analysis. GitHub may trigger existing repository automation on any new branch/PR; review the target repository's workflow behavior before approval. Network failure may leave a branch without a PR; the report records known remote results and retries check for an existing branch/PR. Do not delete remote objects blindly after uncertain failures.

Reports and saved data

Each run stores metadata, evidence and a pinned source snapshot under backend/data by default. Downloads are Markdown, escaped self-contained HTML and a .patch file. Open the HTML locally and use the browser's Print / Save as PDF for a per-run PDF. The milestone PDFs shipped with this project explain development and files, rather than a user's analyzed repository.

History is capped at 20 jobs. Stop the app and move the private data directory to a backup location before starting a fresh history; this preserves recoverability. Keep source snapshots private. .env, caches, data and dependencies are ignored by Git. To relocate data, set an absolute DOCTOR_DATA_DIR in .env. Never share .env or frontend/.env.local.

Troubleshooting

- Backend unavailable: use the exact configured origin, start both services, confirm Java/Maven/Node requirements and that the server tokens match.
- Docker unavailable: static scans still work. Read preflight.sh output; do not bypass the readiness gate.
- Offline dependency failure: prepare a reviewed image cache; the app will not download dependencies during repository execution.
- Root build missing: nested samples can be analyzed, but the current runner only executes the repository root.
- Plan/API failure: inspect Activity, provider configuration and budget limits; start a new run.
- Unsupported Java version: the runtime is Java 21. A newer target is not silently downgraded.

Security boundaries

Host and local API

Single-user local operation is the supported mode. Both services use loopback addresses. Every backend endpoint requires the server token. The Next.js proxy validates Host and browser origin, rejects cross-site requests, limits JSON bodies and proxies an explicit route allowlist. Provider tokens stay server-side. Public hosting requires a separate authentication, authorization, quota and operational-security design.

Untrusted archives are parsed as bounded byte maps, not extracted to host source folders. URLs are restricted to GitHub HTTPS and the code uses fixed GitHub/codeload endpoints without redirects. ZIP paths, duplicates, symlinks/special entries, file counts and expanded sizes are checked. Local persistence contains untrusted data; no host repository shell or Git hook runs. Application dependencies/builds are trusted development work, distinct from inspected repository code.

Repository execution

Docker shares the host kernel; these controls cannot guarantee complete PC safety. Use a dedicated VM for unfamiliar repositories. The controller requires rootless Docker, seccomp, cgroup v2/systemd, a trusted labeled image and no declared image volumes. Every created container rechecks actual limits before importing code.

Containers use an immutable image ID, read-only root, no network, dropped capabilities, no-new-privileges, non-root helper/build identities, 1 CPU, 1,536 MiB memory with no extra swap, 128 process limit, bounded tmpfs and a fixed lifetime. There are no host bind mounts, credentials, SSH agents or Docker sockets inside

repository containers. The helper runs under a distinct UID; build code cannot replace it. Build processes are quiesced before collecting reports. Reports are bounded regular files with XML entity/DTD rejection and symlink/hardlink checks. Tool timeouts and output caps apply. Cancellation removes containers and retries uncertain cleanup.

Static analysis can run without execution prerequisites. The runner never falls back to host commands or relaxes controls. Image labels identify the expected image contract, not cryptographic provenance; operators must build/review their own trusted image. Current base image tags should be replaced with reviewed digests for a distributed release.

Model and publication

Issue/repository text is untrusted model context. Tools and runtime state enforce permissions independently of prompts. The model cannot call a host shell, arbitrary URL, GitHub publish or force-push. Patches require the exact approved file scope, current content digest, unambiguous replacement and size/path checks. Sensitive paths and workflow/hook configuration edits are blocked. Plan approval and publication approval have different digests.

Publishing is constrained to a new codebase-doctor/<UUID> reference and source commit. It never updates default/protected branches and refuses stale source heads or changed reviewed data. The GitHub token belongs to the controller only. Existing remote CI may execute after branch/PR creation; approval authorizes those ordinary GitHub consequences. Test success does not itself authorize publication.

Data and residual risks

Groq receives selected repository context when configured. Known credentials and common secret patterns are redacted from model observations, events and reports, and common sensitive filenames are blocked from model reads. This does not prove all arbitrary embedded secrets are removed. Public repositories may still contain secrets; inspect them before submission. Snapshots deliberately preserve original source bytes locally for honest diffs and therefore may contain such source material. No claim of complete secret scanning is made.

JSON job persistence is private-directory, bounded and atomically replaced, with 20-job retention. Restart invalidates active approvals. This is not a multi-user database or tamper-proof audit log. Source parsers, archive libraries and Docker still need patching and runtime security review. Resource limits reduce denial-of-service risk, not all attacks. Windows/macOS engines and shared/rootful Docker are not supported execution environments for this MVP.

Validation status

Archive/path/patch/approval/report/cgroup-policy behavior is covered by automated tests. Live static source acquisition and browser/API boundaries were checked. Docker daemon access was denied in this session. No actual container isolation, image build, hostile workload containment, live Groq repair or GitHub publication was verified. See VALIDATION.md for evidence.

Architecture and decisions

Data flow

The browser submits repository/mode/objective through the Next.js same-origin proxy. Spring creates a UUID job, retrieves public GitHub metadata and pins the default-branch commit. A bounded ZIP snapshot becomes immutable original bytes plus an edit overlay. JavaParser and build metadata readers produce the source map and findings. If the sandbox is available, the baseline is executed offline; otherwise checks remain NOT_RUN.

Groq receives bounded context and allowlisted tools. Planning has no edit capability. A submitted plan records exact paths and a digest. After approval, repair tools read, search, patch and request verification within fixed budgets. Final verification always attempts a clean snapshot of the changed source. Reports derive facts from saved evidence and actual diffs. A separate publication approval controls GitHub writes.

State and storage

One worker serializes repository work; four active/queued tasks are allowed. Jobs transition QUEUED -> RUNNING -> AWAITING_APPROVAL -> RUNNING -> COMPLETED, or FAILED/CANCELLED. Without Groq, read-only analysis transitions directly to COMPLETED. Completion means the attempt ended, not that builds/tests passed. Job locks protect transitions and patches; snapshots returned to clients are copies. Persistent metadata and source overlays are stored separately with atomic writes. On restart, in-progress jobs fail explicitly and lose approval.

Extension points

LanguageAnalyzer separates source analysis; LlmProvider separates reasoning from Groq transport. The sandbox command policy chooses fixed Maven/Gradle commands. New ecosystems must add parsing, command policy, immutable runtime/helper support and tests rather than exposing shell commands. Persistence can later move behind the store boundary to a database. The current JSON store suits local bounded history.

Important decisions

- Safe source snapshot instead of host clone: no host Git process/hook or checked-out executable scripts. History, submodules and LFS are unsupported. The pipeline's Clone label refers to retrieving a pinned source snapshot.
- Exact byte overlay instead of model shell: approved digest and file scope guard every patch. Git status/diff tools report snapshot changes. They do not claim a local Git branch exists.
- Real remote branch only at publication: branch names are reserved during repair; GitHub Git Data API creates the commit/reference only after a separate approval. Diff exports are standard full-file unified hunks, not minimal Git hunks.
- Offline verification: dependencies are baked into a reviewed image cache. This intentionally limits supported repositories. No untrusted dependency download phase runs on the host.
- Java analysis is syntactic: annotations, packages, class-level relationships and source locations are grounded in JavaParser; no runtime dependency injection graph or database topology is invented. Metadata does not resolve every Maven property/profile or Gradle script.
- CSS components instead of Tailwind/shadcn: the small local UI uses a dedicated stylesheet and typed components to match the owner's portfolio direction without extra dependencies.

- Human-readable job reports are Markdown and printable HTML. Development milestone PDFs are authored separately with full file inventories.

Main modules

repository: bounded HTTPS, archive and path policy. analysis: Java/Spring metadata and symbols. agent: provider, protocol and patch tools. service: workflow and tool dispatch. sandbox: Docker lifecycle and verified offline commands. report: factual Markdown/HTML. github: approved immutable publication. store: private local evidence. api/config: transport and local authorization. Frontend components render these records; no synthetic progress/test data is shipped.

See FILE_GUIDE.md for every authored file, API.md for routes, SECURITY.md for threat boundaries and VALIDATION.md for executed checks.

Local API

Browser requests use /api at http://127.0.0.1:3000. The frontend attaches X-Doctor-Token server-side to the backend at http://127.0.0.1:8080. Direct API clients must provide that token. Never expose it in URLs. POST requests through the frontend require JSON and the exact local Origin. Bodies are capped at 24,000 bytes at the proxy.

Method | Route | Meaning

GET | /api/health | Actual analysis, Docker, Groq and publication readiness

GET | /api/jobs | Recent job summaries; fetch detail before showing full results

POST | /api/jobs | Create job with repository, objective and mode (SCAN/SOLVE)

GET | /api/jobs/{id} | Full immutable response copy of stored job state

GET | /api/jobs/{id}/events | SSE snapshot events and keepalives; reconnect supported

POST | /api/jobs/{id}/approve | Body approvalDigest; authorize the current plan

POST | /api/jobs/{id}/cancel | Interrupt queued/running/awaiting work

GET | /api/jobs/{id}/report | Markdown attachment

GET | /api/jobs/{id}/report.html | Escaped HTML attachment with restrictive CSP

GET | /api/jobs/{id}/diff | Actual unified patch attachment

POST | /api/jobs/{id}/publish | Body publicationDigest and createDraftPr; separate GitHub authorization

Job records include actual events, analysis, plan, baseline/final verification, inspected files, changes, diff, report, concerns, source commit and publication results. Test counts can be null. Status values include NOT_RUN, UNSUPPORTED, SKIPPED, NO_TESTS, UNVERIFIED, PASSED, FAILED and TIMEOUT. Consumers must not convert unknown counts to fabricated totals.

Errors use JSON with a safe message. Invalid input is 400; unknown jobs 404; invalid state/approval is 409; proxy authorization failures 403 and oversized input 413. Provider/network errors are saved on jobs when asynchronous. No endpoint accepts shell commands, arbitrary filesystem paths or arbitrary network destinations.

Milestone delivery status

Updated 20 September 2026. The owner authorized completing all development parts together; no further development approval is required. Product approvals before edits and GitHub writes remain mandatory. The nine consolidated delivery stages below cover the original twelve-stage brief. Each PDF records implementation, file purposes, checks and explicit runtime limits. A historical foundation PDF is retained.

Stage | State | Scope

1. Foundation | Validated | Local UI/API, token guard, setup and initial architecture. Historical PDF preserved.
2. Repository ingestion | Validated | Pinned public GitHub archive, bounded source maps, path policy and persisted workspaces.
3. Java intelligence | Validated | JavaParser metadata, Spring layers, module/symbol maps and evidence-backed findings.
4. Sandbox execution | Implemented; runtime blocked | Rootless Docker controller, immutable guard, offline commands, streaming output and enforced resource checks.
5. Groq planning | Implemented; live provider pending | Groq provider abstraction, structured tools, bounded context, plan generation and workflow state.
6. Approved repair | Implemented; live execution pending | Exact-plan approval, digest-checked scoped patches, true source diff and mandatory final verification.
7. Verification and reports | Reports validated; execution pending | Before/after evidence, principal change, per-file explanations, Markdown/HTML downloads and saved history.
8. GitHub publication | Implemented; live publication pending | Reviewed digest, saved-source consistency, default-head check, new doctor branch and optional draft PR.
9. UI, example and release | Local UI validated; full acceptance pending | Portfolio-inspired UI, broken Spring fixture, complete guides, file inventory and milestone PDFs.

Mapping to the original brief

Original stages 1-6 correspond to delivery stages 1-6. Original stage 7 (human approval) is covered in 6 and 8; stage 8 (verification loop) in 6 and 7; stage 9 (developer UI) in 9; stage 10 (GitHub) in 8; stages 11-12 (example/testing/release) in 9. This is a reporting consolidation, not a claim that unrun external acceptance checks passed.

Acceptance still required

In a dedicated rootless VM, build the reviewed sandbox image, exercise actual resource/cancellation boundaries, configure a Groq key and run the intentionally broken fixture through baseline failure, plan review, approved edit, successful verification and report. GitHub publication additionally needs an authorized test repository/token and explicit in-product approval. No Docker permission changes, key requests or remote writes were performed to manufacture evidence.

Validation record - 20 September 2026

Executed checks

- Backend: **64 tests passed**, no failures/errors. Maven command from backend: `mvn -q -Dmaven.repo.local=../runtime/m2 test`.
- Python: **17 sandbox helper tests + 2 configuration tests passed**. These do not execute external repository scripts.
- Frontend: `npm run build` passed production compilation, TypeScript checks, static page generation and trace collection.
- Local HTTP: **8 API boundary smoke checks passed**: actual readiness/history, invalid intake, cross-origin/Host/Fetch-Site rejection, unknown route and oversized body.
- `docker compose config --quiet` passed configuration parsing only.
- Browser: verified portfolio-inspired dark/light desktop styling, phone layout at 390px without horizontal overflow, saved-history reopening, complete documentation and no console errors observed. Space Grotesk is the rendered heading font; fonts are served locally.

Real source and documentation path

A browser-started scan of `spring-guides/gs-rest-service` completed using source commit `3f4cef01152596c19bc4d37939409358812b1421`. It found 6 Java declarations and recorded Java 17/Maven, multiple Spring versions and nested module evidence. The overview, architecture and downloadable Markdown/HTML were inspected. Baseline build and tests correctly remain `NOT_RUN` because the Docker daemon is inaccessible. There were no source edits or GitHub writes.

Tests are not live provider evidence

Service workflow tests use explicit mock source/Groq/runner boundaries. They check plan approval, patch scope, before/after recording, late cancellation and cleanup on report failure. GitHub tests use a mocked HTTP transport, checking new doctor references, draft PR payloads, source/approval integrity, moved base heads, collisions and duplicate prevention. Sandbox Java tests use a fake Docker transport; Python tests use controlled cgroup/report fixtures. These prove application policy behavior, not real isolation or successful repository execution.

Debugging completed

Corrected `JavaParser/DOM Node` ambiguity, report escaping of generated HTML entities, quoted Git diff paths, saved-run detail loading, cancellation/approval task tracking, publication snapshot consistency and report refresh. API tests now use a temporary store so existing local history cannot affect results. Added bounded line streaming and interruption-safe sandbox cleanup. Reports preserve unknown test counts and distinguish recorded workflow completion from verification success.

Unverified boundaries

`preflight.sh` reports Docker daemon permission denied. No socket permission was changed and no untrusted code ran on the host. The trusted sandbox image has not been built/run here; resource enforcement, cache adequacy, real Groq repair and actual GitHub branch/PR publication remain acceptance tasks. Compose runtime startup is also unverified. Full end-to-end MVP acceptance cannot be claimed until those checks run in the intended environment.

Machine-readable suite results are in `validation-results.json`. Generated milestone PDFs include the relevant checks and limitations rather than substituting mock tests for live execution.

Repository report format and evidence rules

`ReportGenerator.generate(Job)` creates Markdown from the recorded job snapshot. `generateHtml(Job)` wraps the same escaped Markdown text in a self-contained, script-free HTML document suitable for downloading or printing. The HTML intentionally preserves report text rather than interpreting arbitrary repository Markdown.

Sections

- **Run context:** repository, requested objective, state, reserved review branch, pinned source revision, recorded timestamps, and publication metadata when available.
- **Main changes:** text line statistics derived from the actual unified diff and a separately labeled agent explanation. A model-provided explanation is not test evidence.
- **Repository overview:** declared ecosystem versions and modules, snapshot file count, parsed Java types, annotation-supported layers, and structural repository relationships with file/line references. This is a source snapshot, not a runtime call graph; references can precede later edits.
- **File-by-file changes:** actual patch statistics, what changed, and why. A missing diff prevents statistics from being treated as verified. Quoted Git paths, including escaped quotes and octal UTF-8 sequences, are decoded for statistics. Unsupported or malformed headers are disclosed instead of inventing a path. Counts describe the recorded patch, which may use complete-file replacement hunks, rather than the smallest possible edit. Binary changes do not have meaningful text line counts.
- **Before and after verification:** separate baseline/final build/test statuses, counts, duration, and recorded output. Missing runs read `Not run`; missing counts read `Not reported`. Timeouts, failures, and skipped work remain visible. No test count is inferred from an exit code.
- **Static findings:** deterministic source observations separated from suspected issues, each with available evidence and suggested action. Absence of findings is not proof of correctness.
- **Remaining concerns:** errors, cancellation, incomplete checks, explicit concerns, snapshot limits, and redaction limitations.
- **Inspection and patch:** actual file-inspection records, branch/revision context, and the recorded diff. A same-server `/api/jobs/{id}/diff` link appears only for a valid run identifier.

Content safety and size limits

Untrusted text is HTML-escaped and Markdown metacharacters are escaped in one pass so generated entities remain valid. Multiline inline fields cannot inject headings or table rows. Tool output and patches use fences longer than any embedded backtick run. The HTML document escapes the entire report and permits no scripts or remote resources.

Recognizable API-token patterns, named credential assignments, and Authorization values are redacted. This is best-effort redaction, not a guarantee that all secrets are detected; review reports before sharing. Per-command report excerpts stop at 32,768 characters and disclose truncation. Architecture and finding tables show at most 250 entries each and disclose any additional omitted entries. Full available patch text remains in the report subject to the upstream job's own limits.

Static analyzer scope

JavaSpringAnalyzer reads source text as data. It does not execute build scripts or configuration. XML parsing rejects DOCTYPE and external entities. Maven properties, compiler-plugin settings, Spring Boot declarations, literal Gradle versions/toolchains, and literal module includes are recognized. Dynamic Gradle logic and unresolved property inheritance remain unknown.

JavaParser extracts declared classes, records, interfaces, enums, annotations, and nested types. Dependencies are included only when repository types resolve from explicit imports, same-package names, or unambiguous wildcard imports. Generic type parameters do not become repository dependencies. Parsing is structural, not complete Java compiler symbol resolution.

The analyzer accepts at most 2,000 entries, 262,144 characters per file, and 8,388,608 total source characters; omitted entries are disclosed. Unsafe relative paths, null content, and malformed Java cannot silently become valid analysis. Findings currently cover empty catch blocks, Spring field injection, large declared types, and suspected hardcoded credential literals. These rules do not diagnose every bug; the expiry example still requires issue context and real tests.

Validation

Backend unit tests cover Maven/Gradle detection, safe XML rejection, Java type/annotation relationships, ambiguous and generic names, snapshot limits, malformed source, truthful before/after reporting, diff-derived statistics including the quoted paths emitted by SourceTools, untrusted Markdown/HTML, safe fences, redaction, cancellation, and missing evidence. They parse strings and generate reports; they do not execute the deliberately broken Spring fixture.

Complete authored-file tree

Generated runtime files, private configuration values, PDF outputs and dependencies are excluded. The historical Milestone 1 PDF preserves its original delivery inventory.

```
.dockerignore
.env.example
.gitignore
PROJECT_CONTEXT.md
README.md
backend/
  pom.xml
  src/
    main/
      java/
        dev/
          codebasedoctor/
            DoctorApplication.java
          agent/
            GroqLlmProvider.java
            LlmProvider.java
            SourceTools.java
          analysis/
            JavaSpringAnalyzer.java
            LanguageAnalyzer.java
          api/
            ApiErrors.java
            FoundationController.java
          config/
            LocalApiGuard.java
            Redactor.java
          github/
            GitHubPublisher.java
          model/
            Models.java
          report/
            ReportGenerator.java
          repository/
            BoundedHttp.java
            RepositoryPolicy.java
            RepositorySource.java
          sandbox/
            DockerSandbox.java
            SandboxPolicy.java
          service/
            DoctorService.java
          store/
            JobStore.java
        resources/
          application.properties
    test/
      java/
        dev/
          codebasedoctor/
            FoundationApiTest.java
          agent/
            SourceToolsTest.java
          analysis/
            JavaSpringAnalyzerTest.java
          github/
            GitHubPublisherTest.java
          report/
            ReportGeneratorTest.java
          repository/
            RepositoryPolicyTest.java
          sandbox/
            DockerSandboxTest.java
            SandboxPolicyTest.java
          service/
```

```
DoctorServiceTest.java
docker/
  backend.Dockerfile
  frontend.Dockerfile
  guard.py
  sandbox.Dockerfile
  test_guard.py
docker-compose.yml
docs/
  API.md
  ARCHITECTURE.md
  DOCUMENTATION_FEATURE.md
  FILE_GUIDE.md
  MILESTONES.md
  MILESTONE_1_VALIDATION.md
  RELEASE_NOTES.md
  SECURITY.md
  USER_GUIDE.md
  VALIDATION.md
  file-inventory.json
  milestone-data.json
  report-format.md
  validation-results.json
examples/
  broken-spring-app/
    ISSUE.md
    README.md
    pom.xml
    src/
      main/
        java/
          example/
            tokens/
              TokenApplication.java
              TokenController.java
              TokenExpiryService.java
      test/
        java/
          example/
            tokens/
              TokenExpiryServiceTest.java
frontend/
  AGENTS.md
  CLAUDE.md
  app/
    api/
      [...path]/
        route.ts
    globals.css
    layout.tsx
    page.tsx
  components/
    icons.tsx
    job-panels.tsx
    workbench.tsx
  lib/
    client.ts
    types.ts
  next-env.d.ts
  next.config.ts
  package-lock.json
  package.json
  public/
    favicon.svg
    fonts/
      DMMono-Regular.ttf
      Manrope-Variable.ttf
      SpaceGrotesk-Variable.ttf
      dmmono-0FL.txt
      manrope-0FL.txt
      spacegrotesk-0FL.txt
```

```
tsconfig.json
milestone-reports/
  README.md
scripts/
  dev.py
  dev.sh
  generate_reports.py
  preflight.sh
  setup-local.sh
  smoke.py
  test.sh
  test_dev.py
```

Every file and its purpose

`.dockerignore`

Retained foundation file | Delivery stage 1

Excludes secrets, dependencies, build output, and local runtime data from Docker build contexts.

`.env.example`

Updated from foundation | Delivery stage 9

Secret-free server configuration template for local API, Groq, optional GitHub publishing, sandbox image and data location.

`.gitignore`

Retained foundation file | Delivery stage 1

Keeps credentials, installed dependencies, build artifacts, caches, and runtime data out of Git.

`PROJECT_CONTEXT.md`

Updated from foundation | Delivery stage 9

Canonical future-agent handoff: owner instructions, actual code state, architecture, safety, checks and remaining acceptance tasks.

`README.md`

Updated from foundation | Delivery stage 9

Project overview and startup, capabilities, demo, validation boundaries, documentation navigation and current limits.

`backend/pom.xml`

Updated from foundation | Delivery stage 3

Pins Java/Spring build dependencies, JavaParser, Commons Compress and application testing/package configuration.

`backend/src/main/java/dev/codebasedoctor/DoctorApplication.java`

Retained foundation file | Delivery stage 1

Starts the Spring Boot API and discovers application components.

`backend/src/main/java/dev/codebasedoctor/agent/GroqLlmProvider.java`

Created | Delivery stage 5

Sends bounded tool-calling requests to Groq, validates provider responses and stops safely on missing keys or malformed output.

`backend/src/main/java/dev/codebasedoctor/agent/LlmProvider.java`

Created | Delivery stage 5

Provider abstraction separating the agent loop from Groq HTTP transport.

backend/src/main/java/dev/codebasedoctor/agent/SourceTools.java

Created | Delivery stage 6

Reads snapshot files, applies approved digest-checked exact patches and creates true changed-file records and full-file unified diffs.

backend/src/main/java/dev/codebasedoctor/analysis/JavaSpringAnalyzer.java

Updated from foundation | Delivery stage 3

Uses JavaParser plus secure XML/literal build metadata parsing to detect stack, modules, Spring layers, symbols, relationships and findings.

backend/src/main/java/dev/codebasedoctor/analysis/LanguageAnalyzer.java

Retained foundation file | Delivery stage 3

Defines the source-analysis extension boundary for future ecosystems.

backend/src/main/java/dev/codebasedoctor/api/ApiErrors.java

Created | Delivery stage 2

Maps invalid input, missing jobs and invalid workflow states to safe JSON API errors.

backend/src/main/java/dev/codebasedoctor/api/FoundationController.java

Updated from foundation | Delivery stage 2

Exposes health, job lifecycle, approvals, cancellation, SSE snapshots, downloads and separately approved publication through REST.

backend/src/main/java/dev/codebasedoctor/config/LocalApiGuard.java

Retained foundation file | Delivery stage 1

Requires a strong configured API token, authenticates API calls, and rejects oversized requests.

backend/src/main/java/dev/codebasedoctor/config/Redactor.java

Created | Delivery stage 5

Redacts configured tokens and common credential patterns from model observations, logs and reports.

backend/src/main/java/dev/codebasedoctor/github/GitHubPublisher.java

Created | Delivery stage 8

Validates publication approval/source integrity, creates new Git objects and doctor references, and optionally creates an approved draft PR.

backend/src/main/java/dev/codebasedoctor/model/Models.java

Updated from foundation | Delivery stage 2

Defines typed persisted job, event, analysis, plan, change and verification records shared across modules.

backend/src/main/java/dev/codebasedoctor/report/ReportGenerator.java

Created | Delivery stage 7

Generates escaped Markdown and self-contained printable HTML from actual stored analysis, changes and verification evidence.

backend/src/main/java/dev/codebasedoctor/repository/BoundedHttp.java

Created | Delivery stage 2

Fixed-deadline HTTPS transport with redirect rejection, bounded bodies and interrupt handling.

backend/src/main/java/dev/codebasedoctor/repository/RepositoryPolicy.java

Created | Delivery stage 2

Validates GitHub URLs, source paths, text/size limits and sensitive or prohibited edit paths.

backend/src/main/java/dev/codebasedoctor/repository/RepositorySource.java

Created | Delivery stage 2

Pins public GitHub source commits, retrieves same-repo issues and validates ZIP snapshots without host extraction.

backend/src/main/java/dev/codebasedoctor/sandbox/DockerSandbox.java

Updated from foundation | Delivery stage 4

Checks engine/image readiness, launches isolated offline containers, streams bounded output, verifies reports and cleans up after failures/cancellation.

backend/src/main/java/dev/codebasedoctor/sandbox/SandboxPolicy.java

Created | Delivery stage 4

Validates sandbox jobs, immutable image IDs, source import payloads and fixed execution policy.

backend/src/main/java/dev/codebasedoctor/service/DoctorService.java

Created | Delivery stage 5

Coordinates intake, parsing, baseline evidence, bounded model tools, plan approval, repair, cancellation and final reports.

backend/src/main/java/dev/codebasedoctor/store/JobStore.java

Created | Delivery stage 2

Atomically persists private bounded job/snapshot data, creates response copies and invalidates interrupted approvals at restart.

backend/src/main/resources/application.properties

Retained foundation file | Delivery stage 1

Binds the API to loopback, sets request/error-handling limits, and maps environment-based configuration.

backend/src/test/java/dev/codebasedoctor/FoundationApiTest.java

Updated from foundation | Delivery stage 2

Five isolated-store API tests for local authentication, honest readiness, invalid intake, weak startup tokens and oversized requests.

backend/src/test/java/dev/codebasedoctor/agent/SourceToolsTest.java

Created | Delivery stage 6

Checks patch approval/digests, creations/deletions, actual diffs, stale publication review and provider response shape.

backend/src/test/java/dev/codebasedoctor/analysis/JavaSpringAnalyzerTest.java

Created | Delivery stage 3

Thirteen tests for Java/Spring parsing, Maven/Gradle detection, reference grounding, unsafe XML and metadata corner cases.

backend/src/test/java/dev/codebasedoctor/github/GitHubPublisherTest.java

Created | Delivery stage 8

Four mocked HTTP tests verify new doctor branches/draft payloads, retry behavior, stale or tampered state and branch collision rejection.

backend/src/test/java/dev/codebasedoctor/report/ReportGeneratorTest.java

Created | Delivery stage 7

Sixteen regression tests for factual reports, unknown counts, malicious markup, secret redaction and quoted diff paths.

backend/src/test/java/dev/codebasedoctor/repository/RepositoryPolicyTest.java

Created | Delivery stage 2

Five tests reject unsafe URLs, traversal, symlink ZIP entries, oversized sources and prohibited paths.

backend/src/test/java/dev/codebasedoctor/sandbox/DockerSandboxTest.java

Created | Delivery stage 4

Nine fake-transport tests exercise readiness, isolation flags, results, streaming and container lifecycle behavior without executing Docker.

backend/src/test/java/dev/codebasedoctor/sandbox/SandboxPolicyTest.java

Created | Delivery stage 4

Three tests enforce bounded imports, source-path safety and execution policy.

backend/src/test/java/dev/codebasedoctor/service/DoctorServiceTest.java

Created | Delivery stage 5

Five mocked workflow tests cover exact approval, allowed edits, missing sandbox, late cancellation and report-error cleanup.

docker-compose.yml

Updated from foundation | Delivery stage 9

Local static-analysis UI/API deployment with a private named data volume, loopback exposure and no Docker socket mount.

`docker/backend.Dockerfile`

Updated from foundation | Delivery stage 9

Builds the Spring service image and prepares writable private persistence for the unprivileged runtime user.

`docker/frontend.Dockerfile`

Retained foundation file | Delivery stage 1

Builds the frontend with all source directories and runs its standalone production server as a non-root user.

`docker/guard.py`

Created | Delivery stage 4

Immutable container helper: checks enforced limits, imports bounded files, invokes fixed offline builds, quiesces build processes and parses real JUnit reports.

`docker/sandbox.Dockerfile`

Created | Delivery stage 4

Builds the trusted Java/Maven/Gradle execution image with independent non-root helper/build users and a reviewed offline dependency seed.

`docker/test_guard.py`

Created | Delivery stage 4

Seventeen Python tests for cgroup checks, archive import bounds, safe JUnit parsing, offline commands and process/report safety.

`docs/API.md`

Created | Delivery stage 9

Documents authenticated local REST/SSE endpoints, payload meanings, error states and approval boundaries.

`docs/ARCHITECTURE.md`

Created | Delivery stage 9

Explains data flow, extension boundaries, persistence, source-snapshot decisions and implementation tradeoffs.

`docs/DOCUMENTATION_FEATURE.md`

Updated from foundation | Delivery stage 7

Defines the implemented end-user report covering repository overview, principal/per-file changes and verification evidence.

`docs/FILE_GUIDE.md`

Created | Delivery stage 9

Readable complete folder tree and per-file purpose inventory for handoff and maintenance.

docs/MILESTONES.md

Updated from foundation | Delivery stage 9

Tracks nine consolidated delivery stages and their mapping to the original twelve-stage brief.

docs/MILESTONE_1_VALIDATION.md

Retained foundation file | Delivery stage 1

Records exact Milestone 1 checks, debugging fixes, observed results, and unverified limitations.

docs/RELEASE_NOTES.md

Created | Delivery stage 9

Records delivered behavior, debugging and exact runtime acceptance limits for the local MVP.

docs/SECURITY.md

Created | Delivery stage 9

Describes local API, archive, model, sandbox and publishing boundaries, residual risks and validation limits.

docs/USER_GUIDE.md

Created | Delivery stage 9

Gives setup, Groq/Docker configuration, review/publish workflow, reports, data management and troubleshooting steps.

docs/VALIDATION.md

Created | Delivery stage 9

Records executed checks and distinguishes mocked integration evidence from unrun external acceptance checks.

docs/file-inventory.json

Created | Delivery stage 9

Structured authored-file inventory with purpose, principal milestone and creation/modification classification.

docs/milestone-data.json

Created | Delivery stage 9

Machine-readable stage titles, implementation status, checks and limitations used for PDF generation.

docs/report-format.md

Created | Delivery stage 7

Explains the report renderer contract, evidence hierarchy, escaping and exact diff-derived counts.

docs/validation-results.json

Created | Delivery stage 9

Machine-readable final test suite counts, live-scan evidence and explicit runtime blockers.

examples/broken-spring-app/ISSUE.md

Created | Delivery stage 9

Human-readable objective for repairing the bundled token-expiry example.

examples/broken-spring-app/README.md

Created | Delivery stage 9

Documents the intentionally broken fixture, expected bug and safe sandbox-only demonstration procedure.

examples/broken-spring-app/pom.xml

Created | Delivery stage 9

Defines the Java 21/Spring Boot Maven fixture and its JUnit/Spring test dependencies.

examples/broken-spring-app/src/main/java/example/tokens/TokenApplication.java

Created | Delivery stage 9

Boot entry point for the intentionally broken Spring demonstration repository.

examples/broken-spring-app/src/main/java/example/tokens/TokenController.java

Created | Delivery stage 9

Example REST endpoint exposing token-expiry behavior for a realistic repair target.

examples/broken-spring-app/src/main/java/example/tokens/TokenExpiryService.java

Created | Delivery stage 9

Intentionally incorrect expiry boundary used to demonstrate diagnosis and scoped repair.

examples/broken-spring-app/src/test/java/example/tokens/TokenExpiryServiceTest.java

Created | Delivery stage 9

Regression cases specifying the expected expiry behavior; execute only inside the approved sandbox.

frontend/AGENTS.md

Retained foundation file | Delivery stage 1

Next.js-generated development guidance for future coding agents; created automatically by the dev server.

frontend/CLAUDE.md

Retained foundation file | Delivery stage 1

Next.js-generated reference to the local agent guidance file.

frontend/app/api/[...path]/route.ts

Updated from foundation | Delivery stage 9

Guards same-origin browser requests and proxies allowed API/SSE/download routes with server-only credentials and bounded bodies.

`frontend/app/globals.css`

Updated from foundation | Delivery stage 9

Portfolio-inspired theme palette, locally served fonts, floating navigation, editorial hero, result panels and responsive/reduced-motion rules.

`frontend/app/layout.tsx`

Retained foundation file | Delivery stage 1

Defines the page document, application metadata, and global stylesheet import.

`frontend/app/page.tsx`

Retained foundation file | Delivery stage 1

Renders the main Codebase Doctor workbench page.

`frontend/components/icons.tsx`

Created | Delivery stage 9

Reusable local SVG icons for navigation, workflow, downloads and status controls.

`frontend/components/job-panels.tsx`

Created | Delivery stage 9

Renders overview, architecture, findings, plan, activity, terminal, per-file diff and report downloads from real job records.

`frontend/components/workbench.tsx`

Updated from foundation | Delivery stage 9

Interactive UI shell, source form, history hydration, real SSE/polling, plan approval, publication review, theme and navigation.

`frontend/lib/client.ts`

Created | Delivery stage 9

Typed API client, bounded request timeouts, job display helpers and safe GitHub link validation.

`frontend/lib/types.ts`

Updated from foundation | Delivery stage 9

TypeScript contracts for actual API health, job, analysis, findings, plans, verification and publishing state.

`frontend/next-env.d.ts`

Updated from foundation | Delivery stage 1

Provides generated Next.js type declarations for TypeScript.

`frontend/next.config.ts`

Updated from foundation | Delivery stage 9

Configures standalone builds, local compiler checks and browser security headers with production eval disabled.

frontend/package-lock.json

Retained foundation file | Delivery stage 1

Locks resolved frontend dependency versions for repeatable installation.

frontend/package.json

Retained foundation file | Delivery stage 1

Defines Next.js/React/TypeScript dependencies and development, build, start, and type-check commands.

frontend/public/favicon.svg

Retained foundation file | Delivery stage 1

Provides the application-specific browser icon.

frontend/public/fonts/DMMono-Regular.ttf

Created | Delivery stage 9

Locally served DMMono typeface matching the portfolio; avoids browser requests to external font services.

frontend/public/fonts/Manrope-Variable.ttf

Created | Delivery stage 9

Locally served Manrope typeface matching the portfolio; avoids browser requests to external font services.

frontend/public/fonts/SpaceGrotesk-Variable.ttf

Created | Delivery stage 9

Locally served SpaceGrotesk typeface matching the portfolio; avoids browser requests to external font services.

frontend/public/fonts/dmmono-OFL.txt

Created | Delivery stage 9

OFL license and attribution for the bundled dmmono font.

frontend/public/fonts/manrope-OFL.txt

Created | Delivery stage 9

OFL license and attribution for the bundled manrope font.

frontend/public/fonts/spacegrotesk-OFL.txt

Created | Delivery stage 9

OFL license and attribution for the bundled spacegrotesk font.

frontend/tsconfig.json

Retained foundation file | Delivery stage 1

Defines TypeScript compiler checks and frontend import aliases.

milestone-reports/README.md

Updated from foundation | Delivery stage 9

Indexes the historical/new milestone PDFs, complete handbook, source data and regeneration instructions.

scripts/dev.py

Updated from foundation | Delivery stage 9

Safely parses configuration as data and supervises backend/frontend process groups, including compiled standalone UI preview.

scripts/dev.sh

Updated from foundation | Delivery stage 9

Runs idempotent local setup and forwards development/production-preview options to the supervisor.

scripts/generate_reports.py

Created | Delivery stage 9

Reproducibly renders eight later milestone PDFs and the complete handbook from saved facts, guides and the file inventory.

scripts/preflight.sh

Created | Delivery stage 4

Read-only Docker prerequisite checks; never installs services or changes socket permissions.

scripts/setup-local.sh

Updated from foundation | Delivery stage 9

Creates/reuses local random token configuration, rejects inconsistent files and restricts private settings to mode 0600.

scripts/smoke.py

Updated from foundation | Delivery stage 9

Eight live local HTTP checks for readiness, history, invalid intake and browser/proxy authorization boundaries.

scripts/test.sh

Updated from foundation | Delivery stage 9

Runs configuration and guard regression checks, Maven tests and the frontend production build.

scripts/test_dev.py

Retained foundation file | Delivery stage 1

Checks that configuration stays literal and cannot override host command lookup, and that service process groups are terminated.